

AI技术调研与场景分析

课程：软件过程与管理

学生：文就南

学号：2023141461219

案例对象：校园二手交易平台软件过程智能化改进

1. 调研背景

生成式 AI、代码智能体和智能测试平台正在改变软件工程的基本组织方式。传统软件过程强调“阶段活动、角色分工、文档评审、质量门禁”，优点是稳定、可追踪、便于管理；缺点是需求文本整理、设计文档同步、测试用例编写、代码审查和缺陷定位等环节高度依赖人工经验，容易出现重复劳动和反馈滞后。2024 年以来，AI 在软件工程中的应用从单点辅助逐步进入全生命周期：需求阶段可以自动整理访谈纪要、识别冲突与遗漏；设计阶段可以基于需求草案生成架构备选方案和 UML 初稿；编码阶段可以完成代码补全、单元测试生成和局部重构；测试阶段可以根据需求、接口文档和历史缺陷生成测试用例；运维阶段可以基于日志与指标进行异常检测和根因分析。

本报告以“校园二手交易平台”为例进行场景调研与匹配分析。该平台包含用户认证、商品发布、搜索推荐、在线沟通、订单交易、评价举报和后台审核等模块。项目规模适中、需求变化频繁、测试场景较多，适合作为 AI

融入软件过程的课程案例。调研目标不是简单罗列工具，而是判断哪些过程节点最值得引入 AI，并给出可落地的匹配理由。

2. 2024-2026 年 AI 软件工程技术趋势

2.1 需求工程：从“文档生成”走向“需求质量检查”

2024 年的需求工程综述指出，生成式 AI 已被广泛用于需求抽取、需求分类、用户故事生成、验收标准生成和一致性检查，但主要挑战集中在可解释性、可复现性与可控性。对课程项目而言，AI 最适合承担“初稿生成、遗漏提示、冲突扫描、验收标准补全”这类任务，不宜直接替代需求负责人做需求确认。原因在于校园二手交易平台涉及真实用户、交易安全和隐私保护，需求最终仍必须由产品负责人、开发负责人和测试负责人共同确认。

2.2 系统设计：从“画图助手”走向“方案比较器”

AI 可以根据需求说明生成领域模型、组件图、接口草案和数据表初稿，也能解释不同架构方案的取舍。例如“商品搜索”可以选择数据库模糊查询、搜索引擎索引或向量召回；“图片审核”可以选择人工审核、规则过滤或视觉模型辅助审核。AI 的价值在于快速给出多个备选方案和风险提示，帮助团队提前讨论性能、安全、成本和可维护性，而不是只生成一张看似完整的架构图。

2.3 编码实现：从“代码补全”走向“上下文感知开发”

GitHub Copilot、Cursor、Codex

等工具已经从函数级补全发展到仓库级问答、任务拆解、代码修改和测试运行。研究显示，AI 能显著减少样板代码、注释、单元测试和简单重构的时间，但在跨文件复杂任务、业务规则密集任务和安全敏感

代码上仍需要人工审查。校园二手交易平台中，适合 AI 辅助的编码任务包括 DTO/VO 转换、接口校验、CRUD 模板、单元测试、异常处理和日志埋点；不适合完全交给 AI 的任务包括支付/交易状态机、权限控制、隐私脱敏和后台审核策略。

2.4 测试验证：从“用例生成”走向“风险驱动测试”

AI 测试的成熟方向包括需求到测试用例生成、接口测试脚本生成、UI 自动化脚本自愈、缺陷聚类 and 预测性缺陷检测。对于学生项目，最容易落地的是“从需求说明生成测试用例 + 人工评审修正 + 自动化脚本初稿”。AI 可覆盖大量边界条件，例如空输入、重复发布、非法图片格式、价格异常、库存状态不一致、未登录访问等。测试负责人应把 AI 输出作为候选测试资产，而不是最终测试结论。

2.5 运维监控：从“告警通知”走向“根因分析”

运维阶段的 AI 应用主要包括日志聚类、异常检测、容量预测、告警降噪、根因分析和修复建议。校园二手交易平台虽然不是大型商业系统，但仍可设计轻量化 AI Ops 流程：采集接口耗时、错误率、登录失败次数、图片上传失败率、订单状态异常数等指标；当异常发生时，AI 根据日志片段和最近发布记录生成初步分析，运维负责人再确认处理。

3. 过程场景匹配方法

本报告从五个维度评价 AI

场景价值：效率提升、质量提升、落地难度、风险可控性、与现有过程耦合度。每项 1-5 分，综合分越高越适合优先改进。

生命周期环节	候选 AI 场景	效率	质量	落地	风险可控	综合判断
---	---	---	---	---	---	---
需求分析	访谈纪要转用户故事与验收标准	5	4	5	4	优先
需求分析	需求冲突、遗漏与歧义检查	4	5	4	4	优先
系统设计	架构备选方案与接口草案生成	4	4	4	3	推荐
编码实现	CRUD、校验、单元测试初稿生成	5	3	5	4	优先
编码实现	AI 自动合并复杂业务代码	3	2	2	2	暂缓
测试验证	需求驱动测试用例生成	5	5	5	4	优先
测试验证	UI 自动化脚本自愈	3	4	3	3	观察
运维监控	日志异常解释与根因建议	4	4	3	3	推荐

4. 关键匹配场景

场景一：AI 辅助需求建模与验收标准生成

传统过程中的需求分析通常由组员访谈、整理需求说明、开评审会完成。问题在于自然语言需求容易出现边界条件缺失，例如“用户可以发布商品”没有说明图片数量限制、价格范围、重复发布处理、违规词过滤、审核状态变化和下架规则。引入 AI 后，需求负责人先输入访谈纪要和业务目标，AI 生成用户故事、验收标准、异常场景和待确认问题清单。人工评审重点从“从零写文档”转向“确认真实性和优先级”。预期效果是需求文档初稿时间减少 40% 左右，验收标准覆盖率提升。

场景二：AI 辅助需求一致性与可测性检查

课程项目常见问题是需求说明、原型图和测试用例之间不一致。例如需求写“商品发布后直接展示”，测试用例写“商品需管理员审核后展示”，设计文档又写“敏感词命中后进入人工审核”。AI 可作为一种一致性检查员，对需求说明、流程图、接口列表进行交叉检查，输出冲突项、模糊项和不可测项。该场景的价值在于减少后期返工，而不是追求一次生成完美文档。

场景三：AI 辅助接口设计与架构评审

系统设计阶段引入 AI 的重点是“方案比较”。以商品搜索为例，AI 可以从数据量、查询性能、实现复杂度和扩展性比较 MySQL LIKE、倒排索引和向量召回方案；以图片审核为例，AI 可以比较人工审核、关键词规则、视觉模型和混合审核流程。设计负责人根据项目规模选择轻量方案，并把 AI 生成的风险提示纳入设计评审记录。

场景四：AI 辅助单元测试与接口测试生成

测试阶段是本项目最值得优先改进的环节。商品发布、用户登录、订单状态流转都有大量边界条件，人工编写容易漏测。AI 可根据需求、接口契约和数据字典生成等价类、边界值、异常路径和安全性测试用例。测试负责人再筛选、补充、去重，并把高价值用例转化为自动化脚本。该场景具有投入小、可验证、效果直观的特点。

场景五：AI 辅助运维异常解释

平台上线后可采集日志与指标，AI 根据异常窗口内的日志、接口耗时和最近变更记录生成初步根因假设。例如“商品图片上传失败率升高”可能与对象存储配置、图片大小限制、前端压缩失败或网关超时有关。AI 的输出不能直接作为故障结论，但可缩短排查路径，尤其适合课程项目后期演示。

5. 场景优先级结论

综合分析后，本项目建议优先落地三个场景：AI 辅助需求建模、AI 辅助测试用例生成、AI 辅助代码与单元测试生成。其理由是：第一，这三个环节与学生项目现有流程最容易衔接，不需要复杂平台建设；第二，产出物可直接进入评审，包括用户故事、验收标准、测试用例、代码片段和单元测试；第三，风险可通过人工审核、版本管理和质量门禁控制。系统设计和运维监控可以作为第二阶段优化，先以模板化提示词和轻量日志分析方式接入。

AI 融入软件过程的核心不是“让 AI 替代开发者”，而是把 AI 固定为过程中的可管理资源：输入要规范，输出要评审，风险要记录，指标要度量。只有这样，AI 才能从个人技巧变成团队过程能力。

6. 典型工具能力对比

为了避免调研停留在概念层面，本报告进一步从工具能力角度比较常见 AI 软件工程工具。不同工具的定位不同：通用大模型适合需求、文档、评审和解释；IDE 编程助手适合代码补全和局部重构；软件工程智能体适合在仓库中执行明确任务；AI 测试平台适合把需求、接口和历史缺陷转化为测试资产。

工具类别	代表工具	适用阶段	主要优势	主要限制
---	---	---	---	---

| 通用大语言模型 | ChatGPT、Claude、通义千问 | 需求、设计、测试、文档 | 理解自然语言强，适合生成初稿和检查清单 | 容易幻觉，需要人工核验 |

| IDE 编程助手 | GitHub Copilot、Cursor | 编码、单元测试 | 与代码上下文结合紧密，能降低重复编码时间 | 对复杂业务理解有限 |

| 软件工程智能体 | Codex、Devin 类工具 | 编码、修复、测试运行 | 可执行多步任务，能读仓库、改代码、跑测试 | 需要明确任务边界和安全隔离 |

| AI 测试工具 | Testin XAgent、智能测试平台 | 测试验证 | 能生成用例、脚本和缺陷分析 | 与项目接口规范和环境质量相关 |

| AIOps 工具 | 日志分析与告警智能平台 | 运维监控 | 能做异常摘要、根因假设和告警降噪 | 需要足够日志数据和指标体系 |

对学生项目而言，不宜一开始就引入过重的平台。更合理的路线是先使用通用大模型和 IDE 插件完成轻量验证，再把成熟做法固化为提示词模板、检查表和过程节点。如果项目后续扩展为真实系统，再考虑接入专门的智能测试平台和日志分析平台。

7. AI 场景评分矩阵

本报告采用五维评分法进一步细化候选场景。评分不是绝对评价，而是为了帮助团队在有限时间内选择最值得落地的改进点。

场景	效率收益	质量收益	成本复杂度	风险可控性	推荐优先级
---	---	---	---	---	---
需求访谈转用户故事	5	4	1	4	A
验收标准补全	4	5	1	5	A
需求冲突检查	4	5	2	4	A
架构方案比较	3	4	2	3	B
接口文档初稿	4	3	2	4	B
代码补全与样板生成	5	3	1	4	A
单元测试生成	4	4	2	4	A
UI 自动化脚本自愈	3	4	4	3	C
日志根因分析	4	4	3	3	B

评分结果显示，优先级 A 的场景都具有一个共同特点：输入材料明确，输出可人工评审，失败成本较低，且能够直接转化为过程交付物。优先级 C 的场景并非没有价值，而是对环境、工具链和数据积累要求更高，不适合作为课程作业第一阶段重点。

8. 与原软件过程的适配性分析

若原过程已经包含需求评审、设计评审、编码规范、测试计划和部署说明，则 AI 的加入不需要改变过程骨架，只需要在关键活动中增加“AI 辅助输入/输出”和“人工确认规则”。例如需求评审前增加 AI 检查清单，代码提交前增加 AI 辅助审查，测试设计时增加 AI 候选用例生成。这样既能体现智能化改进，又不会破坏软件过程管理课程强调的可控性。

如果把 AI 作为个人临时工具使用，团队很难沉淀经验；如果把 AI 作为过程资产管理，就能形成可复用能力。过程适配的重点包括：提示词模板化、输出版本化、人工评审责任明确化、质量指标可量化、风险问题可追踪。只有这五项同时存在，AI 场景才真正融入软件过程。

9. 适合本项目的 AI 提示词资产

本项目可沉淀以下提示词模板。第一类是需求模板：要求 AI 按角色、目标、前置条件、主成功场景、异常场景、验收标准输出。第二类是设计模板：要求 AI 基于需求列出架构备选方案，并比较复杂度、性能、安全和维护成本。第三类是测试模板：要求 AI 按等价类、边界值、异常输入、权限、安全和状态流转输出测试用例。第四类是审查模板：要求 AI 检查代码中的空指针、越权、注入、日志泄露、重复逻辑和异常处理。

这些提示词不是一次性文本，而是过程资产。每次使用后应记录输入摘要、AI 输出、人工修改点和最终采纳结果。经过多轮迭代后，团队可以逐步形成适合自己项目的提示词库。

10. 调研结论深化

2024-2026 年 AI 软件工程发展的重要变化，是 AI 从“生成内容”走向“参与过程”。早期使用 AI 的方式通常是让模型写一段代码或一篇文档；现在更重要的是让 AI 成为需求分析、设计决策、测试验证和运维排查中的可管理节点。本项目最适合采用渐进式融合路线：第一阶段落地需求和测试，第二阶段扩展到编码和代码审查，第三阶段再尝试运维日志分析。这样的路线既能体现智能化趋势，又能避免过度依赖和不可控风险。

参考文献与资料

1. Cheng H., Husen J. H., Lu Y. 等. Generative AI for Requirements Engineering: A Systematic Literature Review, arXiv, 2024. <https://arxiv.org/abs/2409.06741>
2. Huang Y., Chen Y., Chen X. 等. Generative Software Engineering, arXiv, 2024. <https://arxiv.org/abs/2403.02583>
3. Pandey R., Singh P., Wei R., Shankar S. Transforming Software Development: Evaluating the Efficiency and Challenges of GitHub Copilot in Real-World Projects, arXiv, 2024. <https://arxiv.org/abs/2406.17910>
4. Song F., Agarwal A., Wen W. The Impact of Generative AI on Collaborative Open-Source Software Development: Evidence from GitHub Copilot, arXiv, 2024. <https://arxiv.org/abs/2410.02091>
5. Li H., Zhang H., Hassan A. E. The Rise of AI Teammates in Software Engineering (SE) 3.0, arXiv, 2025. <https://arxiv.org/abs/2507.15003>
6. OpenAI. Introducing Codex, 2025. <https://openai.com/index/introducing-codex/>
7. GitHub Docs. GitHub Copilot documentation, 2025-2026. <https://docs.github.com/en/copilot>
8. Microsoft. Playwright MCP documentation, 2025-2026. <https://playwright.dev/>